bisalli

PAGE NO.....
DATE / /

Subquery.

What is sub Query?

→ subquery is simply an sql query which is placed inside another sql query.

Q: Find the employees who's salary is more than the average salary earned by all employees.

1) find the avg salary
2) filter the employees based on the above result.

Explanation / explaination

1st approach

```
Select avg(salary) from employee; -- 5791.66 (output).

select *
from employee
where salary > 5791.66
```

2nd approach

```
Select *                    # outer query/main query
from employee
where salary(select avg(salary) from employee);
                            # subquery / inner query.
```

employee.

emp_id	emp_name	dppt_name	salary

dhawa dorje ghising

PAGE NO.....
DATE / /

Advantages

(1) we can execute this inner query on its own and it has no dependency on any other query.

(2) outer query is basically using the inner query to filter the records and fetch the final result.

"Different types of Subquery"

(1) Scalar Subquery

(2) Multiple Row Subquery

(3) Correlated Subquery.

1) Scalar Subquery.

→ It always return 1 row and 1 column.

Syntax

```
select *
from table_name [Alias]
join (select avg(selected_col) from table_name) [virtual_table]
on
```

dhawa dorje ghising

PAGE NO.....
DATE / /

Syntax

```
Select *
From [table_name] [table_name Alias]
join ( Select avg [selected_col] [subquery Alias]
       from employee) [virtual_table]
       [table_name]
on [table_name Alias].[selected_col] > [virtual_
   table].[subquery Alias]
```

Implement

```
Select e.*
From employee e
join (select avg(salary) sal From employee) avg_sal
   on e.salary > avg_sal.sal;
```

Explaination

why we need to store [table_name] in [table_name Alias i.e. employee in e because we already have make table in our database?

For,

1) Readability in Complex Queries.
→ An alias like e makes the query cleaner and more concise.

2) Necessary for self-joins.
→ Let's understand with example

Sql

```
Select
a.emp_name As Manager
b.emp_name As Employee
```

dhawa dorje ghising

PAGE NO.....
DATE / /

```
from
  employee a
Join
  employee b
On
  a.emp_id = b.manager_id;
```

Why are Aliases (a and b) Necessary?

1. The same Table is used Twice:

- The table `employee` is being used twice
  - `a` represents the managers
  - `b` represents the employee
- without aliases, SQL wouldn't know whether you're referring to the manager's row or the employee's row.

2) Disambiguating Columns -

- Both instances of the table have the same column names (emp_id, emp_name, manager_id)
- using aliases makes it clear which instance of the table each column refers to.
  - a.emp_name refers to the manager's name
  - b.emp_name refers to the employer's name

Uhava arie ansing

emp_id	emp_name	manager_id
1	Alice	Null
2	Bob.	1
3	Charlie.	1
4	David.	2.

let's say

- Bob (emp_id = 2) has Alice (emp_id = 1) as manager.
- David (emp_id = 4) has Bob (emp_id = 2) as a manager.

Result

Manager	Employee
Alice	Bob.
Alice	Charlie.
Bob	David.

Note! A self-join lets you compare rows within the same table.

dhava dorje ghising

PAGE NO.....
DATE / /

Where we need subQuery Alias?

→ The alias sal stores the value temporarily returned by AVG(salary). It's like giving the result a short meaningful name so we can use it easily later in the query.

How the Join works?

- The subquery is treated like a "virtual table" containing just one column (sal), and virtual table is joined with the employee table
- The Join ensures only those rows where the salary of an employee is greater than the av. salary are included in the results.
- The result of the subquery (avg-sal) is treated as a temporary table with one column, sal
- The Join conditions matches employees in the employee table (e) whose salary is greater than sal (the av salary)
- on e.salary > avg_sal.sal ensures only rows meeting the condition are included in the result.

Role of ON works?

→ without the ON clause, the query would produce a Cartesian product, meaning every row in employee would pair with every row in the subquery's result. The ON clause ensures only rows meeting the condition are joined.

anawadorje.ghising

PAGE NO.....
DATE / /

Treating the subquery as a virtual Table

- The subquery is treated like a virtual table with one column (`sal`) which contains the computed average salary.

- Alias for the virtual Table (`avg-sal`);
  - The subquery is assigned the alias `avg_sal`
    - This allows us to reference the computed value `sal` from the subquery in the main query using `avg-sal.sal`.

does virtual table is empty or consist sal val values?

→ A virtual table, such as the one created by a subquery in SQL, in ~~out~~ not empty - it contains the data produced by the subquery. The values in the virtual table are dynamically generated and exist only for the duration of the query execution.

sql
```
Select avg(salary) sal from employee
```

- Create a virtual table with one row and one column:
  - ↳ Column name: `sal` (the alias for `AVG(salary)`)
  - ↳ Value: The average of all the salaries in the `employee` table

PAGE NO.....
DATE / /

So, the virtual table (avg_sal) contains the computed avg salary, like this:

sal
45000.123

what happens during the Join?

↳ The virtual table (avg_sal) contains this single row with the average salary (sal).

↳ The value (45000.123) is used in the condition e.salary > avg_sal.sal to filter employees from the employee table.

Important Points

→ Virtual tables are not empty. They are populated with the results of the subquery.

→ They only exist temporarily, during the query execution.

→ The values in the virtual table are used for operations such as filtering, aggregation, or comparisons.

Using the Result in a JOIN:

↳ The Join combines this virtual table (avg_sal) with the main table (employee e) based on a condition:

```
Sql
on e.salary > avg_sal.sal
```

dhaval bhoje group

PAGE NO.....
DATE / /

↳ This condition ensures that only those rows from the [employee] table where the [salary] is greater than the average ([sal]) are included in the results.

Why use a subquery with [Join]?

- Dynamic Calculation

↳ The average salary ([sal]) is dynamically calculated from the data in the [employee] table. If the table data changes, the query will still work without needing hard-coded values.

Reusability

↳ The result of the subquery ([avg_sal.sal]) can be used as part of a condition in the [Join]

In summary

↳ The subquery calculates the average salary and stores it as [sal]

↳ The main query compares each employee e's salary ([e.salary]) to this average ([avg_sal.sal]) and returns only those rows where the employee's salary is greater.

Summary

- [Join] : Combines rows from the two sources.
- [ON] : Filters the combination by specifying a condition (e.g. e.salary > avg_sal.sa)

Oluwa Dorje Chising

PAGE NO.....
DATE / /

Why there is dot in e.salary and avg_sal.sal?

```
sql
on   e.salary > avg_sal.sal;
```

→ The dot notation (e.g. e.salary or avg_sal.sal) is used in SQL to specify which table or alias a column belongs to. This is particularly important when.

1) Aliased Tables / Subqueries Are used:

→ if you assign an alias to a table or subquery (e.g. ([employee e] or [(select AVG(salary) sal from employee)] the alias is used as a shorthand to refer to that table or result set.

→ The dot (.) separates the alias (e.g. [e] or [avg_sal]) from the column name (e.g. [salary] or [sal]).

2) Disambiguating Column Names:

→ If multiple tables or subqueries in the query contain columns with the same name, the dot notation is required to specify which table or alias or column belongs to.

Here,

- [e.salary] refers to the computed column ([sal]) from the subquery (aliased as [avg_sal])

bhawa dorje singh

PAGE NO.....
DATE / /

3. Improving Readability.

→ Even when there's no ambiguity, using the dot notation improves clarity by showing exactly which table or subquery a column belongs to.

dhawa dorje ghising

PAGE NO.....
DATE / /

Multiple Row Sub Query

Types

1) subquery which returns multiple column and multiple row.
2) Subquery which returns only 1 column and multiple rows.
~~-- single column, multiple row subquery~~

1) /* Find the employees who earns the highest salary in each department */

```
select dept_name, max(salary)
from employee
group by dept_name;
```

```
Select *
from employee
where (dept_name, salary) in (select dept_name, max(salary)
      from employee
      group by dept_name);
```

Explaination

1. Inner Query:

```
sql
Select dept_name, Max(salary)
from employee
Group by dept_name;
```

dhakal dorje ghising

↳ This inner query produces a "virtual table" (or result set containing pairs of [dept_name] and the highest [salary] in that department.

→ This virtual table will be used in the outer query for filtering.

2. Outer Query:

↳ [where (dept_name, salary) IN]:
This checks if a given [dept_name] and [salary] pair from the [employee] table exists in the virtual table returned by the inner query.

↳ Matching Records:
For each row in the [employee] table, it compares its [(dept_name, salary)] pair to all pairs in the results set of the inner query.

↳ If there's a match, that row is included in the output.

The In Clause

→ Ensures only rows matching the highest salary per department (as calculated by the inner query) are included.

dhawa deep giri

PAGE NO.....
DATE / /

2) -- single column, multiple row subquery

/* Question : Find department who do not have any employee */

```
use employee_database;

select distinct dept_name from employee;

select *
from department
where dept_name not in (select distinct dept_name from
                        employee);
```

1) `select distict dept_name from employee` (subquery).

→ whole list will return only the departments where they have some employees

2) outer query

→ From the department table find only those department who are not in this list then you should be able to get the department which are basically not having any employees.

- The subquery that in this query here, it's returning multiple rows but it's only returning one column only. That's why this subquery is called single column, multiple row subquery.

dhaval durje gosing

Explaination 1

PAGE NO.....
DATE / /

3) Correlated Subquery

→ A subquery which is related to the outer query.

Question: find the employee in each department who earns more than the average salary in that department.

1. Outer Query

```
Select *
from employee e1
where ...
```

↳ This outer query selects all rows (*) from the employee table, aliased as e1

2. Correlated subquery

```
Select avg(salary)
from employee e2
where e2.dept_name = e1.dept_name.
```

↳ This subquery is executed for each row of the outer query.

- e2.dept_name refers to the department of each row being processed in the subquery.
- e1.dept_name refers to the department of the current row from the outer query (IT in this case)

dhaval bhojegohising

PAGE NO.....
DATE / /

- Example Calculation : For the IT department, the employee table has the following rows:

emp-id	emp-name	dept-name	salary
1	Dawa	IT	20,000
2	Ram	IT	30,000

→ The subquery calculates the average salary:

$$\text{Avg(salary)} = \frac{20,000 + 30,000}{2} = 25,000/-$$

3) Comparison in the Outer Query.

a) The outer query compares the salary of the current row (20000) with the average salary (25000) returned by the subquery.

b) Since 20,000 is not greater than 25,000, this row is excluded from the final result.

bhawa dorje grishi

DATE / /

Generalization

→ For every row in the outer query (e1), the subquery (e2) calculates the average salary for the department of that specific row (e1.dept_name)

→ The subquery result (average salary) is then used to compare with the salary of the current row in the outer query.

Key Points

1. Dynamic Matching.

→ The subquery dynamically calculates the av. salary for the department of the current row in the outer query.

2. Role of e1.dept_name and e2.dept_name

- e1.dept_name: represents the department of the current row in the outer query.
- e2.dept_name: ensures the subquery only includes rows from the same department as the current row.

dhawa durje shising

PAGE NO.....
DATE / /

Simplified Analogy

Think of it like this:

- outer Query (e1): A waiter is processing orders table by table.
- subquery (e2): For each table the waiter visits, they calculate the average tip left by all customers at that specific table.
- The waiter compares the tip left by the current customer with the average tip for their table to decide if the customer is a "generous tipper".

Code

```
select *
from employee e1
where salary > (select avg(salary)
                from employee e2
                where e2.dept_name = e1.dept_name
                );
```

dhawa dorje bhutia

PAGE NO.....
DATE / /

Explaination 2

Imagine you have two versions of the same table called: `employee`

1) e1 is the outer query (the main list of employees you are checking).
2) e2 is the subquery (where you calculate the average salary for a specific department).

what this line does:

This line connects the outer query (e1) with the subquery (e2). it says:
"only compare employee who belong to the same department".

Think of it like a classroom

1) Imagine your school has different classrooms (department), like
   - IT (Dept 1)
   - HR (Dept 2)
   - Marketing (Dept 3)

2) You're in the IT classroom (outer query = e1)
   Now, you look around and ask everyone in the IT classroom (subquery = e2).
   - "Hey, what's the average test score for student in ~~out~~ our IT classroom?"

3) You don't care about the test scores of students in the HR or Marketing classrooms.
   - So, you only match IT students with IT students.

dhana dane grising

PAGE NO.....
DATE / /

Simplified Code Meaning.

- For e1 (outer query) : Look at one employee (e.g. Bob, who is in IT).
- For e2 (subquery) : calculate the average salary for all employees in the IT department.
- The `WHERE e2.dept_name = e1.dept_name` ensures you're comparing Bob's salary only to the IT department's average.

If you didn't include `where e2.dept_name = e1.dept_name`, the subquery would calculate the average salary for the entire company instead of ~~testing~~ focusing on just Bob's department. That's not what we went here.

dhawa dombosising

PAGE NO.....
DATE / /

/* Question : Find department who do not have any employees */

```
select *
from department d
where not exist ( seleef 1
                  from employee e
                  where e.dept_name = d.dept_name;
```

→ The goal is to find all department in the `department` table that do not have any employees in the `employee` table.

Breaking down the Query

Outer Query:

```
sql
  select *
  from ~~depart~~ department d
```

- look at all the department in the `department` table
- d is an alias for the `department` table. Think of it like giving it a nick name for convenience.

Sub query

- For each department (`d.dept-name`), check if there are any matching employees in the `employee` table.
- e is a nickname for the `employee` table.
- The condition `e.dept-name = d.dept_name` ensures we are looking for employees in the same department

dhawa dorje ghising

as the current department in the outer query.

sql

where not exists.

- This checks if the subquery does NOT find any employees for a department.
- If no employees are found for a department, the department is included in the result.

Think of it like a school

Imagine your school has a list of all classes (departments) and a list of all ~~er~~ students (employees).

↳ You want to find out which classes don't have any student.

1) Outer Query:

-) look at on class from the list of classes (e.g. "Math")

2) Subquery:

-) check if there's at least one student in the "Math" class.

↳ If no student are found, the subquery returns nothing.

chewa dorie ghising

PAGE NO.....
DATE / /

3. conditions

- If no students exist (NOT EXISTS) keep the "Math" class in the final list.
- If students do exist, skip this class.

What happens behind the scenes.

- For each department in the `department` table:
  - ↳ The subquery looks for employees in the `employee` tables.
  - ↳ if the subquery doesn't find a match (no employee in that department), the department is included in the result

Example:

Tables:

`department` table:

dept_name
IT
HR
Marketing
Security

`employee` table

emp_name	dept_name
Alice	IT
Bob	Marketing
Carol	HR

manwa dorjee ghising

PAGE NO.....
DATE / /

Query Executions

1) For IT : Subquery finds employees - Exists → skip.

2) For HR : Subquery finds employees → Exists → skip.

3) For Marketing : Subquery finds employees → Exists → skip.

4) For Security : Subquery finds no employees → Does NOT Exist
→ include in result.

Final Output

dept_name
Security

dhawa dorje ghising

PAGE NO.....
DATE / /

3) Compare 1 & 2.

```
Select *
from (
    select product-name, sum(price) as total-sales
    from sales
    group by product-name
    ) sales.

join (
    select avg (total-sales) as sales.
    from (
        select product-name, sum (price) as total-sales
        from sales.
        group by product-name
        ) n.

    ) avg-sales.

on sales.total-sales > avg-sales.sales;
```

1. First Subquery (sales)

```
Select product-name, SUM(price) As total-sales
from sales
Group by product-name;
```

- Purpose : Calculates the total sales for each product
- Group by product-name: Groups rows by each product-name
- sum(price) : Add up the prices for all rows in each product group.

dhawa dorje gnising

PAGE NO.....
DATE / /

Result (Example):

product-name	total-sales
Apple	300
Banana	120
Orange	210

2. Second Subquery (avg-sales).

```
select Avg (
    ----
    ----
    ----
    ----
) n;
```

- Purpose: Calculate the average total sales across all products.
- Inner query (n): The same as the first subquery, giving the `total-sales` for each product.
- AVG(total-sales): Computes the average of these totals.

Example Calculations:

- If the `total-sales` values from the first subquery are: `300 (Apple)`, `120 (Banana)` and `210 (Orange)`
- Then: $\text{avg-sales} = \frac{300 + 120 + 210}{3} = 210$.

chawa dorje ghising

PAGE NO.....
DATE / /

Result:

Sales (avg_sales)
210

3. The JOIN

```
ON  sales.total_sales > avg_sales.sales;
```

- Purpose: Filters and combine rows ~~have~~ where [total_sales] for a product is greater than average total sales.
- sales.total_sales: Refers to the total sales from the 1st subquery
- avg_sales.sales: Refers to the average total sales from the second subquery.

Final Output:

The query produces a result containing only those product whose [total_sales] is greater than the average total sales.

Example Output:

Based on the data:

product_name	total_sales	avg_sales
Apple	300	210

Unauthorized copying

PAGE NO.....
DATE / /

Why the code works?

↳ The inner queries calculate intermediate values. (total sales per product and the average total sales).

↳ The Join applies the conditions (on sales.total_sales > avg_sales.sales) to filter and combine rows.

↳ The query efficiently finds products performing better than the average.

Q) Why there is two alias 'x' and 'avg_sales' on second query?

Role of 'x'!

- The inner query (x) is producing a derived table that calculates total_sales for each product_name.

→ Without an alias, the outer query wouldn't be able to refer to the derived table. SQL requires every derived table to have a name (alias). This alias x allows the outer query to refer to the results of this subquery (x).

Role of avg_sales!

→ The entire second query (which calculates the average total sales) needs to be given an alias (avg_sales) so that it can be referenced in the main query.

dhanu dorje chising

PAGE NO.....
DATE / /

~~Different SQL Clause~~
~~Where subquery is allowed~~

→ The main query uses this alias to compare `sales.total_sales` > `avg_sales.sales` in the on condition of the `Join`

Why two aliases?

→ SQL Syntax Requirement: Every derived table or subquery must have an alias in SQL.

→ Alias `T1`: Used for the inner subquery that calculate `total_sales` by product.

→ Alias `avg_sales`: Used for the outline subquery that calculates the average of `total_sales`.

↳ Aliases allow you to give meaningful names to intermediates steps, making the query easier to write, understand and reference.

without aliases (why it wouldn't work).

1) SQL would throw a error because derived tables must have names.

2) It would become difficult to reference or distinguish between different subqueries in the main query.

dhaval chaughning

PAGE NO.....
DATE / /

In summary

- 'p' is for the inner subquery (total sales by product)
- avg_sales is for the outer subquery (average of total sales).

Each serves a purpose in organizing and ~~running~~ the query structure.

dhawa dorje ghising

PAGE NO.....
DATE / /

Different SQL clause where subquery is allowed

-- using a subquery in select clause. (usually not recommended)

/* question: fetch all employee details and add remarks to those employees who earn more than the average pay */

1. SELECT

~~b) This retrieves all columns (*~~

b) This retrieves all columns (*) from the employee table.

b) You're adding an additional computed column called remarks, to each row.

2. CASE statement

→ The case statement is used to create a conditional expression. Let's break it down.

1) when salary > (select Avg (salary) from employee).

b) This checks if the salary of the current row is greater than the average salary of all employees.

b) The subquery (select AVG (salary) from employee) calculate the average salary from the employee table.

dhawa garje shising

b.) Then 'Higher than average'

↳ If the condition (salary > average salary) is True, the result for that row is 'Higher than average'

c) ELSE NULL

↳ If the condition is False (i.e. the salary is not higher than the average), the result for that row is Null

d) As remarks

↳ This names the computed column remarks

3. FROM employee

↳ This indicates that the query is working on the employee table, applying the logic for every row in this table.

Final Output

For every employee in the tables

1. All the original columns are displayed (due to select *)

2. A new column called remarks is added

↳ If the employee's salary is higher than the average, it contains 'Higher than average'

PAGE NO.....
DATE / /

↳ otherwise, it contains [Null]

Example walkthrough.

suppose this is the [employee] table

emp_id	emp_name	salary
1	Alice	60,000
2	Bob	40,000
3	Carol	50,000.

Avarage salary = (60,000 + 40,000 + 50,000) / 3
= 50,000

output:

emp_id	emp_name	salary	remarks
1	Alice	60,000	Higher than avarage
2	Bob	40,000	Null
3	Carol	50,000	Null.

Key Concepts

- Subquery: (select AVG(salary) from employee) dynamically calculates the avarage salary of all employees.
- CASE statement: Add conditional logic to determine whether the salary is above the avarage

dhawa dorje ghising

PAGE NO.....
DATE / /

- Null: Acts as a place holder for rows that do not satisfy the conditions.

Sql

```
select *
, (case when salary > (select avg(salary) from employee)
        then 'Higher than average'
        else null
        end) as remarks
from employee;
```

dhawa dorje ghising

PAGE NO.....
DATE / /

-- Having

/* Question : Find the store who have sold more units than the average units sold by all stores.

```
Select store_name, sum(quantity)
from sales
Group by store_name
having sum(quantity) > (select avg(quantity) from
                        sales);
```

1. `Select product_name, SUM(quantity)`

L) Retrieve two things for each product

1. `product_name` : The name of the product.
2. `SUM(quantity)` : The total quantity of that product sold (aggregated from the `sales` table)

2. `From sales`

L) Specifies that the data source for this query is the `sale` table

3. `Group BY product_name`.

L) Groups the rows in the `sales` table by `product_name`

L) For each product, it calculates the `SUM(quantity)` separately.

anava drive driving

PAGE NO.....
DATE / /

4.) Having SUM (quantity) > (select AVG (quantity) from sales).

↳ Having Clause

→ Similar to a where clause but used for filtering aggregated data after the Group by operation.

- Conditions
  - filters out groups (i.e. product) where the total quantity (sum (quantity)) is less than or equal to the average quantity of all rows in the sales tables
  - The subquery (select Avg (quantity) from sales) calculates the average quantity across all rows in the sales table

Subquery: (select AVG (quantity) from sales)

- Computes the average quantity across the entire sales tables.
- This value is compared against each product's total quantity (SUM (quantity)) in the having clause

dhaval bhai ghiang

PAGE NO.....
DATE / /

How it Works

1. Subquery Executions.

→ First, the subquery calculates the average quantity from the `sales` table (let's call it `avg_quantity`)

2. Grouping and Aggregation:

→ The outer query groups the `sales` table by `product_name` and calculates the total quantity (`SUM(quantity)`) for each product.

3) Filter with `HAVING`

→ Only includes products where their total quantity (`SUM(quantity)`) is greater than the average quantity (`avg_quantity`)

Example Data:

Suppose the `sales` table contains

product_name	quantity
A	10
B	20
C	30
A	15
B	25

drawn dorje ghising

PAGE NO.....
DATE / /

Step1: Subquery (~~AS~~ AVG (quantity))

- Avaneage quantity = (10 + 20 + 30 + 15 + 25) / 3 ≈ 20.

Step2: Grouping and Aggregations

- After [Group by product-name] and [SUM(quantity)]:

product-name	SUM(quantity)
A	25
B	45
C	30

Step3: Having Clause:

- only product with [SUM(quantity) > 20] are included.

product-name	SUM(quantity).
A	25
B	45
C	30.

Final Output.

product-name	SUM(quantity).
A	25
B	45
C	30

dhawa dorje ghising

PAGE NO.....
DATE / /

Key Points

1. `GROUP BY` organizes the data into groups (per product in this case).

2. `Having` filters aggregated data after the `Group by` operation

3. The sub query calculates the average (`AVG(quantity)`) and serves as a benchmark for filtering.

dhawa dorje ghising

PAGE NO.....
DATE / /

Practical of Primary and foreign key.

```
use employee_database;

# creating Forgein key table.

create table city (
    cid int not null auto_incramant,
    city ~~varchar~~ varchar(50) not null,
    primary key (cid)
    );

# inserting data in city table

insert into city(city)
values
('Agra')
('Delhi')
('Bhopal')
('Jaipur')
('Noida')

# create primary key table.

Create table personal (
    id int not null,
    name varchar(50) not null,
    cityname int not null,

    primary key(id), # making 'id' primary key

    foraign key (cityname) refernances city (cid)
    );
```

dhawa dorje ghising

PAGE NO.....
DATE / /

inserting data in personal table.

```
insert into personal ( id, name, cityname )
values
( 1, "dawa", 1 ),
( 2, "yogita", 2 ),
( 3, "sharad", 3 ),
( 4, "Madhu", 4 ),
( 5, "dawa", 1 ),
( 6, "giti", 2 ),
( 7, "Shar", 3 ),
( 8, "Madho", 5 );
```

displaying two table in same table.

```
select * from personal p inner join city c
on p.cityname = c.cid.
```

here we have extra column like cityname and cid, we have to remove it.

```
select p.id, p.name, c.city
from personal p join city c
on p.cityname = c.cid.
where c.city = "Agra"
order by p.name;
```

dawa dongre gnising